



Haskell Curry, the mathematician whom the language was named after

What is Functional Programming?

One example that's oft cited is that of formulas in Microsoft Excel. When entering formulas in a cell, the final value is always expressed in terms of the values of other existing cells. No attempt is made to specify the order of evaluating these cells; instead, we expect that Excel will figure out dependencies on its own. Similarly, no attempt is made at managing the memory used by the expression. Finally, we program using expressions rather than instructions - an expression merely establishes the relationship between the inputs and the outputs without explicitly listing the steps

required to fulfil that relationship.

As the name suggests, functions in a Functional Programming language (including Haskell) are first class citizens. This means that they can not only be invoked and called like their imperative counterparts, but also that functions can be passed as arguments to other functions, and can even be defined within the scope of other (parent) functions. This feature allows nearly all programming constructs to be expressed in the form of functions.

A hallmark feature of all functional programming languages is that they are said to be 'lazy'. So it only evaluates an expression when it is required, and not before. This is highly advantageous, especially when dealing with compound expressions (expressions where the operands themselves are expressions) or with infinitely large data structures such as infinite lists. Hence, these languages are far more memory efficient than their 'strict evaluation' counterparts. Furthermore, lazy evaluation reduced the number of computations by storing the result of all function calls against their parameters in the form of a lookup table which is used the next time the same function is called without bothering to re-compute it.

Take the example of computing the entire Fibonacci sequence. In Haskell, the sequence can be computed using the following one-liner:

- `fibs = 0 : 1 : zipWith (+) fibs (tail fibs)`

Here, 'fibs' is the name of the list that will store the sequence, '0 : 1' is the initialization of the list, and the 'zipWith' operator performs an operation (in this case, addition) on the corresponding elements of the two operators following it. The first operator is the list fibs itself, while the tail operator